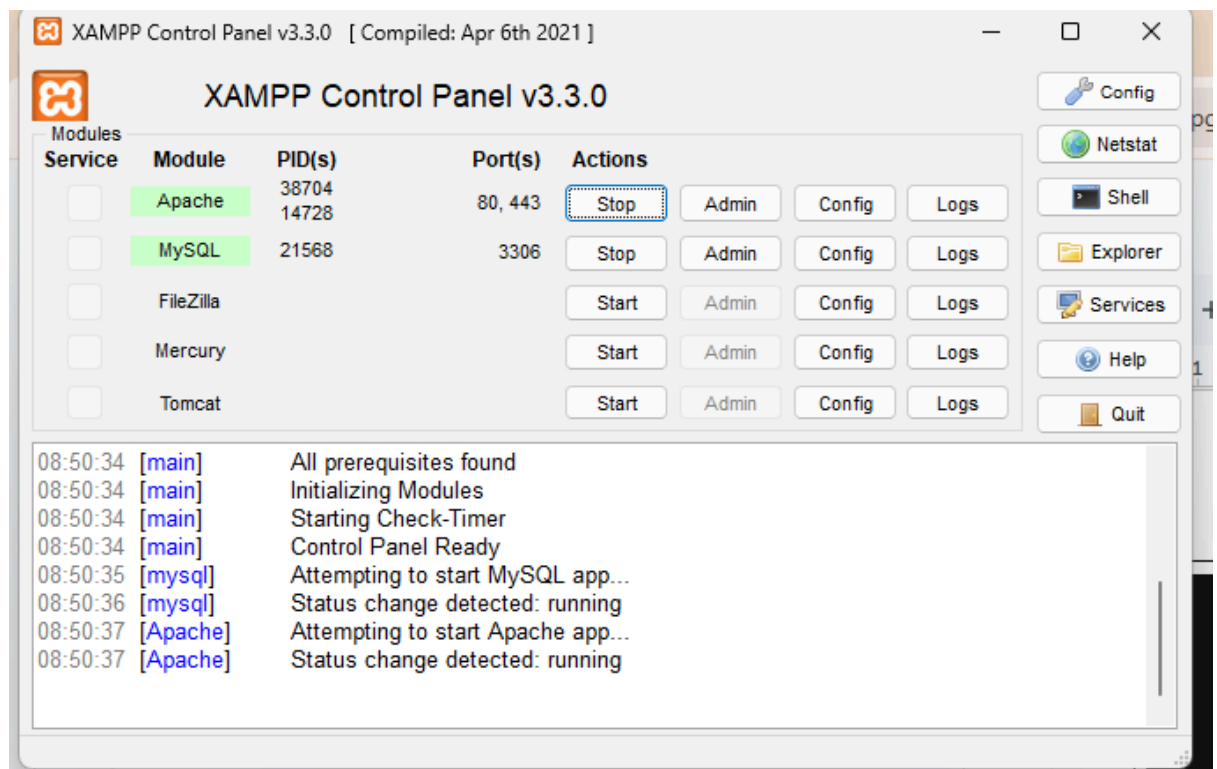


Guida pratica all'uso di database all'interno di un ambiente Java

1. Creazione DB in SQL

1. Scaricare XAMPP e avviare MySQL e Apache



2. Cercare localhost sul browser e accedere a phpMyAdmin

3. Creazione del DB relazionale e tabelle in phpmyadmin

```
CREATE DATABASE IF NOT EXISTS ...
```

```
CREATE TABLE IF NOT EXISTS ... (...);
```

2. Connessione IntelliJ IDE a XAMPP

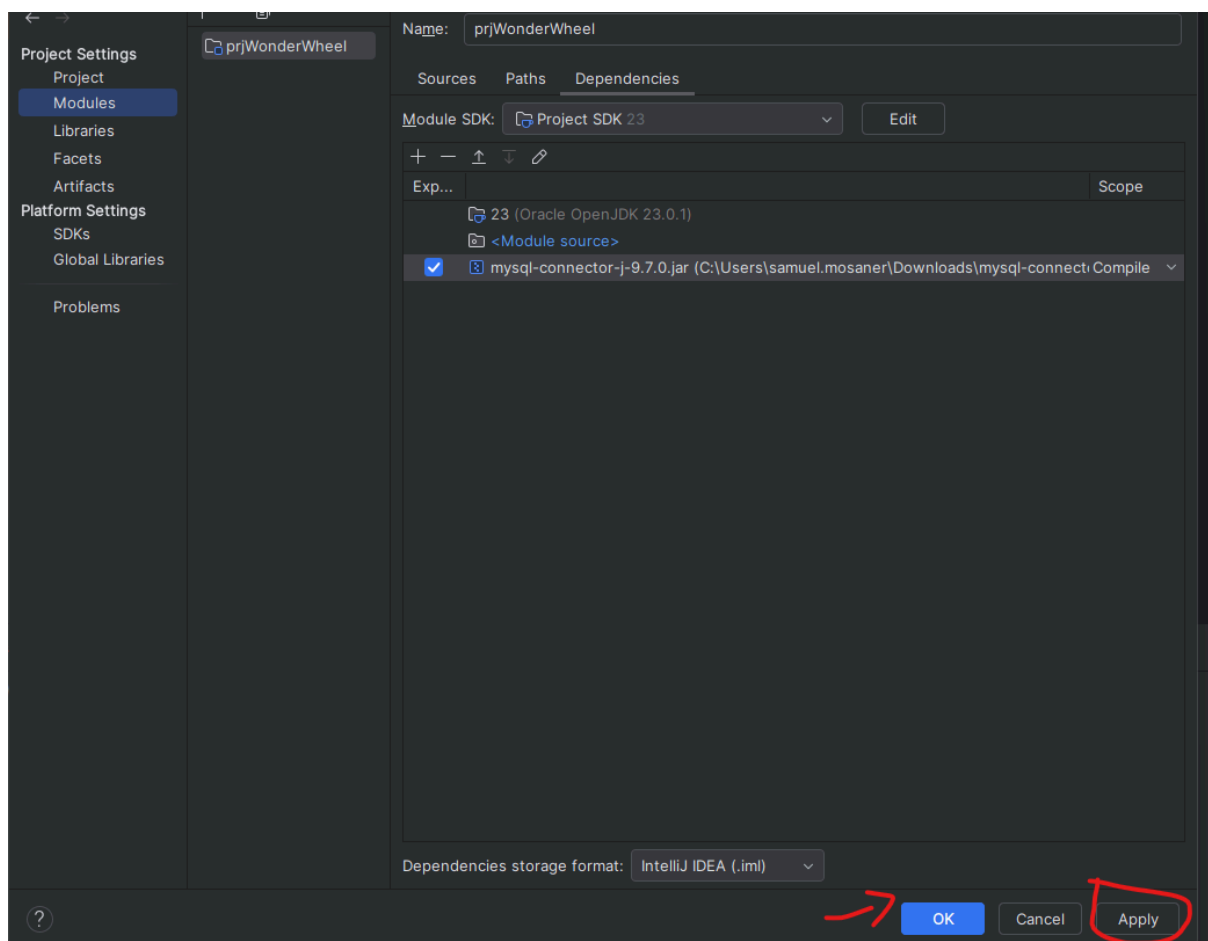
1. Scaricare MySQL Connector/J dal sito ufficiale ([MySQL Connector/J](https://dev.mysql.com/downloads/connector/j/))



2. Aprire Project Structure in IntelliJ e inserire il Connector

Inserire in Modules -> Dependencies -> + il file .jar

Poi cliccare apply e dopo ok



3. Creare nel progetto java la classe Dbconnection

```
//importare queste librerie
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DbConnection {
    //inserire url del DB
    private static final String URL = "jdbc:mysql://localhost:3306/nomeDB";
    //inserire nome user
    private static final String USER = "root";
    //inserire password se inserita
    private static final String PASSWORD = ""; // o la tua password
    //metodo per ritornare il tipo di connessione
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

4. creare metodi CRUD in Java

importare la libreria import java.sql.*

creare una classe 'nomeClasseDAO' e creare all'interno tutti i metodi CRUD e se necessari altri metodi che implementino la query desiderata all'interno del programma.

Dimostrazione pratica con un progetto già completamente implementato.

```
public void inserisci(Persona p) {
    //stringa del comando SQL da eseguire
    String sql = "INSERT INTO pazienti (NomePersona) VALUES (?)";
    //mi connetto al database
    try (Connection conn = DbConnection.getConnection();
        //rappresenta la query in SQL, dove '?' sono i campi
        PreparedStatement ps = conn.prepareStatement(sql)) {
        //inserisco il primo '?'
        ps.setString(1, p.getNome());
        //eseguo la query
        ps.executeUpdate();
        //messaggio di riuscita del comando
    }
```

```

        System.out.println("Paziente inserito: " + p.getNome());

    } catch (SQLException e) {
        //lancia eccezione se non riesce a connettersi al DB
        e.printStackTrace();
    }
}

```

```

// SELECT ALL - restituisce tutte le persone
public List<Persona> trovaTutti() {
    //creo lista dei record
    List<Persona> lista = new ArrayList<>();
    //stringa del comando SQL da eseguire
    String sql = "SELECT * FROM persona";
    //mi connetto al database
    try (Connection conn = DbConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            lista.add(new Persona(
                //aggiungo record alla lista
                rs.getString("NomePersona"),
                rs.getString("id_persona")

            ));
        }

    } catch (SQLException e) {
        //lancia eccezione se non riesce a connettersi al DB
        e.printStackTrace();
    }

    return lista;
}

```

```

public void aggiornaNome(int id, String nuovoNome) {

    //stringa del comando SQL da eseguire
    String sql = "UPDATE pazienti SET NomePersona = ? WHERE
id_persona = ?";

    //mi connetto al database

```

```

    try (Connection conn = DbConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        //aggiorna il record con il nuovo nome
        ps.setString(1, nuovoNome);

        //cerca il record con campo 'id'
        ps.setInt(2, id);

        //esegue la QUERY
        ps.executeUpdate();

    } catch (SQLException e) {
        //lancia eccezione se non riesce a connettersi al DB
        e.printStackTrace();
    }
}

```

```

public void elimina(String id) {

    //stringa del comando SQL da eseguire
    String sql = "DELETE FROM pazienti WHERE id_persona = ?";

    //mi connetto al database
    try (Connection conn = DbConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        //elimina il record con campo 'id'
        ps.setString(1, id);

        //esegue la QUERY
        ps.executeUpdate();

    } catch (SQLException e) {

        //lancia eccezione se non riesce a connettersi al DB
        e.printStackTrace();
    }
}

```

La classe DAO serve solo ad inserire i dati all'interno del DB, per la logica e la struttura dei dati ci deve essere sempre una classe Java non collegata al DB.

I metodi DAO vengono chiamati poi da una classe Java che implementa sia la logica sia il collegamento al DB (esempio clinica medica)

Abbiamo implementato sia la classe Persona che la classe PersonaDAO, come abbiamo implementato la classe Visita e la classe VisitaDAO

```
import java.util.*;

public class ClinicaMedica {
    private Queue<Visita> visite;
    private Deque<String> record;
    private Queue<Visita> urgenti;
    private ArrayList<Visita> visiteProcessate;
    private ArrayList<Persona> pazienti;
    private int maxRecord;

    private PersonaDAO personaDAO;
    private VisitaDAO visitaDAO;

    public ClinicaMedica(int maxRecord) {
        this.visite = new ArrayDeque<>();
        this.record = new ArrayDeque<>();
        this.urgent i = new ArrayDeque<>();
        this.visiteProcessate = new ArrayList<>();
        this.pazienti = new ArrayList<>();
        this.maxRecord = maxRecord;
        this.personaDAO = new PersonaDAO();
        this.visitaDAO = new VisitaDAO();
    }

    public void addPaziente(Persona p) {
        if(!pazienti.contains(p)) {
            pazienti.add(p);
            aggiungiRecord("Aggiunta di un paziente");
            personaDAO.inserisci(p);
        } else {
            throw new PazientiExceptions("Il paziente è già stato
inserito");
        }
    }

    public void addVisita(Visita v) {
```

```

        if(v.getPriorita()){
            urgenti.add(v);
        } else {
            visite.add(v);
        }
        aggiungiRecord("Visita aggiunta correttamente");
        visitaDAO.inserisci(v);
    }

    public void removeVisita(Visita v){
        if (!visiteProcessate.contains(v)) {
            if (visite.contains(v)) {
                visite.remove(v);
            } else {
                urgenti.remove(v);
            }
            aggiungiRecord("Rimozione avvenuta con successo");
            visitaDAO.elimina(v.getDataOra(),
v.getPersona().getId());
        } else {
            throw new VisitaProcessataException("Visita già
processata");
        }
    }

    public void aggiungiRecord(String mes){
        if(record.size() <= maxRecord){
            record.add(mes);
        } else {
            record.removeFirst();
            record.add(mes);
        }
    }

    public Visita processaVisita() {
        Visita temp;

        if (!urgenti.isEmpty()) {
            temp = urgenti.poll();
            visiteProcessate.add(temp);
            aggiungiRecord("Visita urgente processata");
            visitaDAO.processaProssimaVisita();
        }
    }

```

```

        return temp;
    } else if (!visite.isEmpty()) {
        temp = visite.poll();
        visiteProcessate.add(temp);
        aggiungiRecord("Visita processata");
        visitaDAO.processaProssimaVisita();
        return temp;
    }

    throw new VisitaProcessataException("Nessuna visita in coda
da processare");
}

public String recordToString() {
    return record.toString();
}

public String personeToString() {
    String str = "";
    for(int i = 0; i < pazienti.size(); i++){
        str += pazienti.get(i).toString();
    }
    return str;
}

public String visiteProcessateToString() {
    String str = "";
    for(int i = 0; i < visiteProcessate.size(); i++){
        str += visiteProcessate.get(i).toString();
    }
    return str;
}

public ArrayList<Persona> getPazienti() {
    return pazienti;
}

public String visiteInCodaToString() {
    return visite.toString();
}

public Queue<Visita> getUrgenti() {

```

```
        return urgenti;
    }

    public Queue<Visita> getVisite() {
        return visite;
    }
}
```

5. Creazione del Main con menu annesso

1. il Main rimane quasi invariato, solo che bisogna aggiungere i campi che mancano dalla classe di gestione, non inserire il comando in toto